



COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	SECTION:	CPE21S1
DATE PERFORMED:	July 28, 2026	DATE SUBMITTED:	August 1, 2026
NAME:	Benj Federic C. De Leon	INSTRUCTOR:	Engr. Jimlord M. Quejado

ACTIVITY NO. 4.1 Stacks

1. PROCEDURE OUTPUT

ILO A: Create a stack using the C++ STL

Definition from the official CPP documentation for the stack STL

Code:

```
#include <iostream>
#include <stack> // calling stack from the STL

int main(){
    //create an object for the stack
    std ::stack<int> newStack;

    std:: cout<<"Stack Empty? "<<newStack.empty()<<std::
endl;

    //push to the stack
    newStack.push(3);
    newStack.push(8);
    newStack.push(15);

    std:: cout<<"Stack Empty? "<<newStack.empty()<<std::
endl;

    std:: cout<< "The size if the stack:
"<<newStack.size()<<std::endl;

    std:: cout << "The top of the element is: "<<
newStack.top()<< std:: endl;

    std :: cout <<"Removing elements from the stack: \n";
    newStack.pop();

    std:: cout<< "The top of the element is: "<<
newStack.top()<< std:: endl;
```

	<pre> newStack.pop(); std::cout<< "The top of the element is: "<< newStack.top()<< std::endl; newStack.pop(); std::cout<<"Stack Empty? "<<newStack.empty()<<std:: endl; return 0; } </pre>
Output:	<pre> Stack Empty? 1 Stack Empty? 0 The size if the stack: 3 The top of the element is: 15 Removing elements from the stack: The top of the element is: 8 The top of the element is: 3 Stack Empty? 1 </pre>

ILO B.

Implementation of stack through an array and a linked list.

B.1. Stacks using Arrays

Tasks:

Modify the code given above to include a function that will display all elements in the stack.

Provide a description of each operation provided.

Include your output in section 6

Code:

```
#include <iostream>

//static array, what do we need?
const size_t maxCap = 100;
int stack[maxCap];
int top = -1, i, newData, choice;

void push();
void pop();
void Top();
bool isEmpty();
bool isFull();
void all_Stack();

int main(){
    std::cout<<"Give the number of max elemenets for the new
stack: ";
    std::cin>> i;

    while(true){
        std::cout<<"\nStack operations:"<<std::endl;
        std::cout<< "1. PUSH\n2. POP\n3. TOP\n4. isEmpty\n5.
Show all elements of the stack\n" << std::endl;
        std::cin>> choice;

        switch (choice){
```

```

        case 1: push();
        break;
        case 2: pop();
        break;
        case 3: Top();
        break;
        case 4: std::cout<< isEmpty()<<std::endl;
        break;
        case 5: all_Stack();
        break;
        default: std::cout<<"Invalid choice"<<std:: endl;
        break;

    }

}

return 0;
}

bool isEmpty(){
// create a condition to check whether the top element is
equal to -1
    if (top == -1) return true;
    return false;
}

bool isFull(){
//check if we reached the maximum capacity of the array:
if(top >= i - 1) return true;
return false;
}

void push(){
//check if full -> if yes return an error
if(top == i-1){
    std:: cout << "Stack overflow\n";

```

```
        return;
    }

    std::cout<< "New Value to add into the stack"<<std::endl;
    std::cin >> newData;
    //increment the top(index by 1) then assign the new data;
    stack[++top]= newData;
}

void Top(){
    //check if the stack is empty
    if(isEmpty()){
        std:: cout<< "Stack is Empty."<<std:: endl;
        return;
    }

    std:: cout<<"The element on the top of the stack is " <<
stack[top]<<std::endl;
}

void pop(){
    //check if empty -> if yes, return error
    if(isEmpty()){
        std::cout << "Stack Underflow." << std::endl;
        return;
    }

    //display the top value
    std::cout << "Popping: " << stack[top] << std::endl;
    //decrement top value from stack
    top--;
}

void all_Stack(){
    //print all elements
    if(isEmpty()){
        std::cout << "Stack is Empty." << std::endl;
```

	<pre> return; } for (int j = top; j >= 0; j--) { std::cout << stack[j] << " "; } std::cout << std::endl; } </pre>
Output:	<pre> Stack operations: 1. PUSH 2. POP 3. TOP 4. isEmpty 5. Show all elements of the stack 1 New Value to add into the stack 1 Stack operations: 1. PUSH 2. POP 3. TOP 4. isEmpty 5. Show all elements of the stack </pre>

```
1
New Value to add into the stack
2

Stack operations:
1. PUSH
2. POP
3. TOP
4. isEmpty
5. Show all elements of the stack

1
New Value to add into the stack
3

Stack operations:
1. PUSH
```

Code analysis:

This C++ code creates a safe menu-driven stack using an array of up to 100 items. It fixes bugs by adding a menu exit option, stopping bad input sizes, and moving variables inside functions to prevent crashes. It perfectly tracks your items and stops errors when the stack gets too full or completely empty.

B.2. Stacks using Linked Lists

Tasks:

Modify the code given above to include a function that will display all elements in the stack.

Provide a description of each operation provided.

Include your output in section 6.

INPUT:

```
#include<iostream>
#include "ilo_bp2_LL.h"

int main(){

    std::cout<<"--- PUSHING
'c' 'p' 'e' ONTO THE STACK
--- \n";

    push('c');
    push('p');
    push('e');

    displayAll<char>();

    Top<char>();

    std::cout<<"\n -----
Popping all the elements
----- \n";

    while(!isEmpty<char>()){
        std::cout<<"Popped:
"<< pop<char>() <<std::endl;
    }

    std::cout<<"\n -----
Testing the Underflow -----
\n";
    pop<char>();

#endif STACK_LIST_H
#define STACK_LIST_H

template <typename T>
class Node{
public:
    T data;
    Node* next;
};

template <typename T>
Node<T> *head = nullptr;

template <typename T>
void push(T newData){

    Node<T>* newNode = new
Node<T>;
    newNode->data= newData;

    newNode->next = head<T>;
    head<T> = newNode;
}

template <typename T>
T pop(){

    if (head<T> == nullptr)
        std::cout<< "Stack
```



```
return 0;
}
```

```
Overflow \n";

Node<T>* temp = head<T>;

T tempVal = temp->data;

head<T> = head<T>->next;

delete temp;

return tempVal;

}

template <typename T>
void Top() {

    if(head<T> == nullptr) {
        std::cout<<"\n-----
Stack Underflow
-----\n"<<std::endl;
        return;
    }

    std::cout<<"Top of Stack:
" <<head<T>->data
<<std::endl;
}

template<typename T>
bool isEmpty() {
    return head <T> ==
nullptr;
}

template<typename T>
```

```
void displayAll() {  
  
    if(head<T> == nullptr){  
  
        std::cout<<"The Stack  
is Empty\n";  
        return;  
    }  
  
    std::cout<<"\n---  
Displaying the Stack ----\n";  
    Node<T>* currentNode =  
head<T>;  
  
    while(currentNode !=  
nullptr){  
  
        std::cout<<  
currentNode->data << std:::  
endl;  
        currentNode =  
currentNode-> next;  
    }  
    std::cout << "\n";  
}  
  
#endif
```

OUTPUT:

```

--- PUSHING 'c' 'p' 'e' ONTO THE STACK ---

---- Displaying the Stack ----
e
p
c

Top of Stack: e

----- Popping all the elements -----
Popped: e
Popped: p
Popped: c

----- Testing the Underflow -----
Stack Overflow
PS C:\Users\TIPQC\Documents\CPE 010-CPE21S1\HoA4p1\ILOB2>

```

Code analysis:

This C++ code implements a stack using STL, pushing 'c', 'p', and 'e' so that 'e' ends up on top (LIFO order). It displays the stack, shows the top element, then pops all items one by one until empty. Finally, it tries popping from the empty stack to test underflow, which correctly outputs "Stack Overflow".

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using an implementation of stack:

The following problem definition and algorithm is provided for checking balancing of symbols. Create an implementation using stacks. Your output must include the following:

- Stack using Arrays
- Stack using Linked Lists
- (Optional) Stack using C++ STL

INPUT:

```

#include <iostream>
#include "ilo_c.h"

int main() {
    std::string expr;

```

```

#ifndef STACK_STL_H
#define STACK_STL_H

#include <iostream>
#include <stack>

```

```
std::cout << "Enter an
expression to check: ";
std::getline(std::cin,
expr);

if (isBalanced(expr))
    std::cout << "The
symbols are Balanced.\n";
else
    std::cout << "The
symbols are NOT
Balanced.\n";

return 0;
}
```

```
bool isOpening(char c) {
    return (c == '(' || c ==
'{' || c == '[');
}

bool isClosing(char c) {
    return (c == ')' || c ==
'}' || c == ']');
}

bool isMatchingPair(char
open, char close) {
    if (open == '(' && close
== ')') return true;
    if (open == '{' && close
== '}') return true;
    if (open == '[' && close
== ']') return true;
    return false;
}

bool isBalanced(std::string
expr) {
    std::stack<char> s; //
Step 1

    for (int i = 0; i <
expr.length(); i++) { //
Step 2
        char c = expr[i];

        if (!isOpening(c) &&
!isClosing(c)) {
            continue; // a)
ignore non-symbols
```

```

    }

    if (isOpening(c)) {
        s.push(c); // b)
push opening symbol
    } else if
(isClosing(c)) { // c)
closing symbol
        if (s.empty()) {
            std::cout <<
"Error: Unmatched closing
symbol '" << c << "'\n";
            return
false;
        }
        char openChar =
s.top();
        s.pop();
        if
(!isMatchingPair(openChar,
c)) {
            std::cout <<
"Error: Mismatched symbols
'" << openChar
            <<
"' and '" << c << "'\n";
            return
false;
        }
    }

    if (!s.empty()) { //
Step 3
        std::cout << "Error:
Unmatched opening symbol(s)

```

	<pre> remain\n"; return false; } return true; } #endif </pre>
OUTPUT:	<pre> PS C:\Users\Benj Federic De Leon\Documents\Code\ \CPE 010-CPE21S1\HoA4p1\" ; if (\$?) { g++ i Enter an expression to check: (A+B)+(C-D) The symbols are Balanced. PS C:\Users\Benj Federic De Leon\Documents\Code\ \CPE 010-CPE21S1\HoA4p1\" ; if (\$?) { g++ i Enter an expression to check: ((A+B)+(C-D) Error: Unmatched opening symbol(s) remain The symbols are NOT Balanced. PS C:\Users\Benj Federic De Leon\Documents\Code\ \CPE 010-CPE21S1\HoA4p1\" ; if (\$?) { g++ i Enter an expression to check: ((A+B)+[C-D]) The symbols are Balanced. PS C:\Users\Benj Federic De Leon\Documents\Code\ \CPE 010-CPE21S1\HoA4p1\" ; if (\$?) { g++ i Enter an expression to check: ((A+B)+[C-D]) Error: Mismatched symbols '(' and ']' The symbols are NOT Balanced. </pre>

Code analysis:

(A+B)+(C-D) and **((A+B)+[C-D])** are correctly marked Balanced, while **((A+B)+(C-D)** triggers "Unmatched opening symbol(s) remain" for its missing closing parenthesis, and **((A+B)+[C-D]** triggers "Mismatched symbols '(' and ']'" for its incorrect pairing. This matches the expected results in the Self-Checking table.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT



I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Claude

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Code assistance, grammar, formal sentences, provide a full guide idea.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: I try to understand and evaluate the code before actually proper utilizing it.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.